API Kontar para parceiros (ERP)

Documento para o desenvolvedor do ERP que vai integrar com o Kontar. A API é REST/JSON. Cada cliente do Kontar tem seu próprio token e só enxerga os próprios balanços.

URL base

```
https://southamerica-east1-kontar-e2650.cloudfunctions.net/api/v1
```

Todas as rotas abaixo são relativas a essa URL (nos exemplos, `$BASE`). Corpos de requisição são JSON (`Content-Type: application/json`, até 10 MB). Datas são ISO 8601 em UTC.

Autenticação

Todas as rotas, exceto `GET /ping`, exigem o cabeçalho:

```
Authorization: Bearer <token>
```

O token é gerado pelo administrador do cliente no app Kontar: menu **Integrações** → **Gerar novo token**. O token aparece uma única vez; gerar um novo invalida o anterior. Sem cabeçalho ou com token inválido a resposta é `401 nao_autorizado`.

Toda chamada autenticada é registrada (rota, status, tempo) e aparece em "Últimas chamadas" na aba Integrações.

Convenção de erros

Qualquer erro devolve:

```
{ "erro": "codigo", "mensagem": "texto legível" }
```

HTTP	erro	Quando
400	`json_invalido`	Corpo não é JSON válido
400	`balanco_invalido`	`POST /balancos` com campos faltando/inválidos
400	`patch_invalido`	`PATCH` sem nada para atualizar ou com valor inválido

HTTP	`erro`	Quando
400	`catalogo_invalido`	`itens` vazio, acima de 5.000, ou item sem `codigo` e sem `ean`
400	`parte_invalida`	`parte` / `total` fora de 1..100 ou `parte` > `total`
401	`nao_autorizado`	Token ausente ou inválido
404	`nao_encontrado`	Rota inexistente ou balanço não encontrado (ou de outro cliente)
409	`balanco_encerrado`	`PUT catalogo` em balanço encerrado
409	`balanco_aberto`	`GET contagem` / `POST contagem/confirmar` em balanço ainda aberto
409	`nao_editavel`	`PATCH /balancos/{id}` em balanço que não é `origem: "erp"` e `status: "aberto"`
409	`endereco_em_uso`	`PATCH` remove endereço que já tem bipagem
409	`setor_em_uso`	`PATCH` tenta remover setor (setores só podem ser adicionados)
409	`conflito_refErp`	`POST /balancos`: o `refErp` conflita com outro balanço
409	`partes_incompletas`	Última parte do catálogo chegou mas faltam partes ou o `total` diverge
500	`interno`	Erro interno

Objeto Balanço

Devolvido por `GET /balancos`, `GET /balancos/{id}`, `POST /balancos` e `PATCH /balancos/{id}`:

```
{
  "id": "b-1",
  "nome": "Balanço Geral Loja 1",
  "loja": "1",
  "status": "aberto",
  "origem": "erp",
  "tipo": "geral",
  "sequencial": false,
  "enderecos": ["C01", "C02"],
  "setores": [],
  "criadoEm": "2026-10-29T14:00:00.000Z",
  "encerradoEm": null,
  "erp": {
    "refErp": "BAL-1001",
    "catalogoEm": null,
    "catalogoItens": null,
    "contagemEnviadaEm": null,
    "contagemRecebidaEm": null,
    "erro": null
  },
```

```
  "totais": { "bipagens": 0, "enderecos": 2 }
}
```

- `status`: `aberto`, `pausado` ou `encerrado`. `origem`: `erp` ou `kontar`. `tipo`: `geral`, `parcial` ou `surpresa` (`surpresa` só existe em balanços criados no app).
- `totais.bipagens` só é contado em `GET /balancos/{id}`; na listagem e nas demais rotas vem `0`.

Rotas

GET /ping (pública)

Verifica se a API está no ar. Não exige token.

```
curl https://southamerica-east1-kontar-e2650.cloudfunctions.net/api/v1/ping
```

Resposta 200 (resposta real):

```
{"ok":true,"em":"2026-10-29T14:00:06.326Z"}
```

GET /eu

Identifica o cliente dono do token.

```
curl -H "Authorization: Bearer $TOKEN" $BASE/eu
```

Resposta 200 (resposta real):

```
{"clienteId":"loja-teste","nome":"Loja Teste"}
```

GET /balancos?loja=&status=

Lista balanços do cliente. Filtros opcionais: `loja` (código da loja) e `status` (`aberto` ou `encerrado`).

```
curl -H "Authorization: Bearer $TOKEN" "$BASE/balancos?loja=1&status=aberto"
```

Resposta 200:

```
{ "balancos": [ { "id": "b-1", "nome": "Balanço Geral Loja 1", "loja": "1", "status": "aberto",
```

```
"origem": "erp", "...": "ver Objeto Balanço" } ] }
```

POST /balancos

Cria um balanço de origem ERP. Idempotente por `refErp`: se já existe um balanço com esse `refErp`, devolve o existente.

Corpo:

Campo	Tipo	Obrigatório	Observação
`refErp`	string	sim	Identificador do balanço no ERP
`nome`	string	sim	
`loja`	string	sim	
`tipo`	string	não	`geral` (padrão) ou `parcial`
`sequencial`	boolean	não	Precisa ser `true` / `false` de verdade (string ou número dá `400`). Padrão `false`
`enderecos`	string[]	não	Duplicados e vazios são descartados
`setores`	string[]	não	Idem

```
curl -X POST -H "Authorization: Bearer $TOKEN" -H "Content-Type: application/json" \
  -d '{"refErp":"BAL-1001","nome":"Balanço Geral Loja
1","loja":"1","tipo":"geral","sequencial":false,"enderecos":["C01","C02"]}' \
  $BASE/balancos
```

Respostas: `201` com o Objeto Balanço (criado agora), `200` com o Objeto Balanço (já existia), `400 balanco_invalido`.

GET /balancos/{id}

Devolve o Objeto Balanço, com `totais.bipagens` preenchido. `404 nao_encontrado` se não existir.

```
curl -H "Authorization: Bearer $TOKEN" $BASE/balancos/b-1
```

PATCH /balancos/{id}

Altera um balanço. Só vale para `origem: "erp"` e `status: "aberto"`; caso contrário `409 nao_editavel`.

Corpo (informe ao menos um): `enderecos` (string[]), `setores` (string[]), `sequencial` (boolean), `nome` (string não vazia).

- `enderecos` substitui a lista. Endereço que já recebeu bipagem não pode ser removido: `409 endereco_em_uso` (a mensagem lista os endereços).
- `setores` só pode crescer: omitir um setor existente dá `409 setor_em_uso`. Com setores informados o balanço passa a organizar por setores e endereços.
- `sequencial` precisa ser boolean, senão `400 patch_invalido`.

```
curl -X PATCH -H "Authorization: Bearer $TOKEN" -H "Content-Type: application/json" \
  -d '{"enderecos":["C01","C02","C03"]}' $BASE/balancos/b-1
```

Resposta 200: Objeto Balanço atualizado.

PUT /balancos/{id}/catalogo

Envia o catálogo (itens com estoque, custo e venda) usado na contagem. Substitui o catálogo anterior. `409 balanco_encerrado` se o balanço está encerrado.

Corpo:

```
{
  "itens": [
    { "codigo": "1001", "ean": "7891000100103", "descricao": "ARROZ 5KG", "un": "un", "estoque": 10,
"custo": 21.5, "venda": 29.9 }
  ]
}
```

Campo do item	Observação
`codigo` / `ean`	Ao menos um é obrigatório (`400 catalogo_invalido` indicando o número do item)
`descricao`	Cortada em 120 caracteres
`un`	Convertida para maiúsculas
`estoque`, `custo`, `venda`	Opcionais. Devem ser números JSON ou strings com vírgula decimal sem separador de milhar (ex.: `4.10` ou `"4,10"`); `"1.234,56"` NÃO é aceito

Limites: até **5.000 itens por chamada**; `itens` vazio é rejeitado.

Chamada única (até 5.000 itens):

```
curl -X PUT -H "Authorization: Bearer $TOKEN" -H "Content-Type: application/json" \
```

```
  -d @catalogo.json $BASE/balancos/b-1/catalogo
```

Resposta 200:

```
{ "itens": 1, "blocos": 1, "hash": "3f5a...e1", "inalterado": false }
```

Idempotência: o catálogo é comparado por hash do conteúdo. Reenviar o mesmo catálogo devolve `200` com `"inalterado": true` e não regrava nada.

Em partes (catálogos maiores que 5.000 itens): envie `parte` e `total` no corpo, ambos inteiros de 1 a 100, com `parte <= total` (senão `400 parte_invalida`). Cada parte tem no máximo 2.000 itens (acima disso, `400 catalogo_invalido`).

- Partes `1..total-1`: resposta `202` `{ "parte": 1, "total": 3, "aguardando": true }`. O catálogo ainda não foi alterado.
- Última parte (`parte == total`): o servidor confere que existem exatamente as partes `1..total`, todas com o mesmo `total`. Se sim, junta tudo na ordem das partes e responde `200` como na chamada única. Se não, `409 partes_incompletas`: reenvie a partir da parte 1.
- Reenviar uma parte já enviada a substitui. A parte 1 apaga qualquer envio anterior em andamento; no `409 partes_incompletas` as partes guardadas são descartadas e o envio recomeça da parte 1.

```
curl -X PUT -H "Authorization: Bearer $TOKEN" -H "Content-Type: application/json" \
  -d '{"parte":1,"total":3,"itens":[ ... ]}' $BASE/balancos/b-1/catalogo
```

GET /balancos/{id}/contagem?porEndereco=1

Devolve a contagem final. Só existe quando o balanço está `encerrado`; aberto dá `409 balanco_aberto`. O parâmetro `porEndereco=1` é opcional.

```
curl -H "Authorization: Bearer $TOKEN" "$BASE/balancos/b-1/contagem?porEndereco=1"
```

Resposta 200:

```
{
  "itens": [
    { "codigo": "1", "ean": "", "descricao": "A", "un": "UN", "quantidade": 10, "estoque": 10,
"diferenca": 0, "naoCadastrado": false }
  ],
  "totais": { "itens": 1, "unidades": 10, "naoCadastrados": 0 },
  "porEndereco": [
    { "endereco": "C01", "itens": [ { "codigo": "1", "ean": "", "quantidade": 10 } ] }
  ],
```

```
  "balancoId": "b-1",
  "loja": "1",
  "encerradoEm": "2026-10-29T18:00:00.000Z"
}
```

- `quantidade` é o contado; `estoque` vem do catálogo; `diferenca` = contado - estoque.
- `naoCadastrado: true` indica produto bipado que não está no catálogo (`descricao` vem vazia).
- `porEndereco` só aparece com `porEndereco=1`. Ele **exclui as correções** (`_CORRECAO`) feitas na auditoria, portanto a soma por endereço de um produto pode ser menor que a `quantidade` total dele em `itens`. A quantidade oficial é sempre a de `itens`.

POST /balancos/{id}/contagem/confirmar

O ERP avisa que recebeu a contagem. Sem corpo. `409 balanco_aberto` se o balanço não está encerrado.

```
curl -X POST -H "Authorization: Bearer $TOKEN" $BASE/balancos/b-1/contagem/confirmar
```

Resposta 200:

```
{ "ok": true, "contagemRecebidaEm": "2026-10-29T18:05:00.000Z" }
```

PATCH /balancos/{id}/erp

O ERP informa seu estado de processamento, que aparece no balanço (bloco `erp`). Informe ao menos um campo, senão `400 patch_invalido`:

Campo	Efeito
`erro`	string: registra a mensagem de erro (até 500 caracteres) e a hora; `""` ou `null` limpa o erro
`contagemEnviadaEm`	`true` : registra que o ERP gravou a contagem no seu lado
`pedirCatalogo`	`false` : baixa o pedido de catálogo feito pelo usuário no Kontar

```
curl -X PATCH -H "Authorization: Bearer $TOKEN" -H "Content-Type: application/json" \
  -d '{"contagemEnviadaEm":true}' $BASE/balancos/b-1/erp
```

Resposta 200: `{ "ok": true }`

O que o ERP precisa implementar

Fluxo mínimo, com o balanço criado no Kontar (origem Kontar):

1. **Listar balanços abertos da loja:** `GET /balancos?loja=<loja>&status=aberto`. Repita periodicamente (por exemplo, a cada minuto).
2. **Enviar o catálogo** de cada balanço aberto: `PUT /balancos/{id}/catalogo` com os itens e o estoque/custo/venda da loja. Em até 5.000 itens, uma chamada; acima disso, em partes. Reenviar é seguro (`inalterado: true`).
3. **Consultar o status até `encerrado`:** `GET /balancos/{id}` e observar `status`.
4. **Buscar a contagem e confirmar:** `GET /balancos/{id}/contagem` (com `?porEndereco=1` se o ERP aceitar local), gravar no ERP, depois `POST /balancos/{id}/contagem/confirmar`. Opcionalmente `PATCH /balancos/{id}/erp` com `contagemEnviadaEm: true` ou `erro`.

Alternativa, com o balanço nascendo no ERP: no lugar do passo 1, o ERP cria o balanço com `POST /balancos` informando `refErp` (idempotente, pode repetir sem duplicar) e mantém endereços/setores atualizados com `PATCH /balancos/{id}`. Os passos 2 a 4 são iguais.

Perguntas ao ERP parceiro

1. Em qual tabela fica o balanço aberto (número, loja, data, status, tipo geral/parcial, sequencial)? Existe tabela de endereços do balanço e recorte de itens para o balanço parcial?
2. Qual tabela e layout o ERP lê como contagem de inventário (nome, colunas obrigatórias, campo de status/lido, se aceita local/endereço)?
3. Qual coluna de estoque por loja na tabela de itens, e quais colunas de custo e venda por loja?
4. Qual usuário MySQL (e permissões) o conector pode usar para ler essas tabelas e gravar a contagem?

Se a pergunta 1 não tiver resposta (não existe balanço aberto no ERP), todo balanço nasce no Kontar e o ERP apenas envia o catálogo.